

**UNIVERSIDADE FEDERAL DE OURO PRETO
DEPARTAMENTO DE COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

MARCELO VIEIRA MAGALHÃES

RELATÓRIO: TRABALHO PRÁTICO II

Leitura e Seleção de Certificados do Windows via Aplicação Web, Extensão de Navegador e Host Nativo

Disciplina: Criptografia e Segurança de Redes
(BCC423)

Professor: Dr. Carlos Frederico Marcelo da Cunha
Cavalcanti

Ouro Preto – MG

Sumário

1	Introdução e Contexto	2
2	Descrição da Arquitetura Proposta	2
3	Explicação da Comunicação Extensão – Host Nativo	2
4	Leitura de Certificados no Windows (CryptoAPI)	3
5	Discussão sobre Segurança e Limitações	4

1 Introdução e Contexto

Sistemas modernos de segurança digital dependem fortemente da infraestrutura de chaves públicas (PKI) e, especificamente, do uso de certificados digitais no padrão X.509 para garantir confidencialidade, autenticidade e não-repúdio em transações e comunicações eletrônicas. Tais certificados encontram-se comumente instalados e protegidos diretamente no sistema operacional ou em repositórios controlados do ecossistema do usuário.

Contudo, aplicações web convencionais executadas no escopo restrito de um navegador enfrentam limitações severas de segurança. Por questões críticas de privacidade e isolamento de processos (*sandboxing*), o motor JavaScript de uma página web comum não possui privilégios de acesso aos recursos nativos de hardware ou ao armazenamento local de certificados do sistema operacional.

Com o intuito de mitigar essa limitação técnica sem comprometer a segurança, este relatório descreve a implementação de uma solução modular de software desenvolvida para o Trabalho Prático II (TP2). O sistema estabelece uma infraestrutura segura de acesso e seleção de certificados X.509 presentes no repositório pessoal do Windows (`CurrentUser\My`), preparando o ecossistema para futuras operações criptográficas no escopo do Trabalho Prático III (TP3).

2 Descrição da Arquitetura Proposta

A solução foi arquitetada sob um modelo em camadas independentes e fortemente desacopladas, dividindo-se em três principais componentes estruturais de software:

1. **Aplicação Web / Interface do Usuário** (`popup.html` / `popup.js`): Uma interface gráfica minimalista construída com tecnologias web puras (HTML5, CSS3 e JavaScript), encarregada de exibir a tabela de certificados ao usuário de forma tabular e intuitiva, permitindo interações de seleção única ou em lote.
2. **Extensão de Navegador (Google Chrome – Manifest V3)**: Atua como a ponte de comunicação segura do sistema. Utilizando APIs internas de mensagens, recebe as requisições geradas pelo *popup* e as encaminha ao ambiente do sistema operacional, blindando a página web de qualquer contato direto com binários locais.
3. **Componente Nativo / Proxy Local** (`main.cpp`): Um programa compilado nativamente em C++ para a plataforma Windows. Possui as permissões operacionais necessárias para se comunicar com as APIs criptográficas do núcleo do sistema operacional, atuando como um barramento que lê o cofre de certificados e traduz os metadados brutos em mensagens estruturadas no formato JSON.

O fluxo informacional e a disposição lógica desses componentes são representados de maneira simplificada na Figura 1.

3 Explicação da Comunicação Extensão – Host Nativo

A comunicação bidirecional estabelecida entre a Extensão do Google Chrome e o componente executável local baseia-se estritamente no protocolo oficial **Native Messaging** da Google Chrome API.

Esse mecanismo utiliza os canais de entrada e saída padrão do sistema operacional (`stdin` e `stdout`) operando obrigatoriamente sob o modo binário estruturado. Cada pacote de dados trafegado deve conter um cabeçalho fixo composto de exatamente 4 bytes precedendo a mensagem, o qual codifica um número inteiro de 32 bits correspondente ao tamanho exato (em bytes) do conteúdo textual subsequente formatado em UTF-8 JSON.

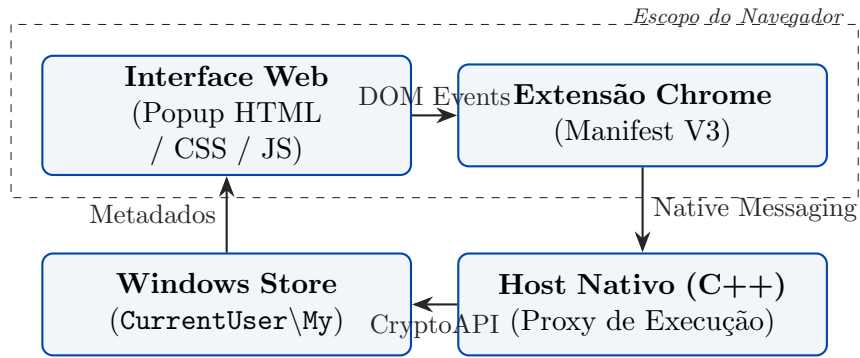


Figura 1: Diagrama de Arquitetura em Camadas do Sistema.

No escopo da extensão (`popup.js`), a conexão persistente é aberta por meio do método `chrome.runtime.connectNative("com.tp.certlist")`. Uma mensagem contendo a string de comando `{"text": "GET_CERTS"}` é disparada pelo canal aberto.

Do lado nativo, a aplicação C++ monitora o `stdin` em um laço de repetição contínuo. Ao ler o cabeçalho e constatar o tamanho da requisição recebida, decodifica a estrutura, inicia a rotina de varredura no repositório de segurança e escreve de volta no canal `stdout` o comprimento da string de resposta, seguido imediatamente do vetor JSON contendo a matriz de metadados extraída dos certificados locais.

4 Leitura de Certificados no Windows (CryptoAPI)

Para realizar a leitura física das estruturas criptográficas e preencher os Requisitos Funcionais mínimos exigidos (RF1), o programa em C++ consome a biblioteca nativa **CryptoAPI** (`wincrypt.h`) do ecossistema Windows, sendo linkado em tempo de compilação com a biblioteca `Crypt32.lib`.

A rotina operacional inicia-se com a invocação da função `CertOpenStore`, parametrizada para abrir o provedor de sistema `CERT_STORE_PROV_SYSTEM` sob o escopo contextual de usuário logado `CERT_SYSTEM_STORE_CURRENT_USER`, apontando especificamente para o repositório lógico de certificados pessoais de codinome `L"MY"`.

Uma vez obtido o manipulador (*handle*) estável da memória do cofre, executa-se um processo iterativo por meio do método `CertEnumCertificatesInStore`. A cada ciclo do laço de iteração, um ponteiro estruturado do tipo `PCCERT_CONTEXT` é retornado, disponibilizando o acesso aos metadados mapeados nos requisitos:

- **Subject (Assunto) e Issuer (Emissor):** Extraídos textualmente da estrutura de dados via chamada de `CertGetNameStringW`, utilizando os seletores de exibição simples e amigável.
- **NotBefore (Início) e NotAfter (Expiração):** Capturados na subestrutura interna do registro do cartão de identificação (`pCertInfo`) e convertidos do padrão nativo `FILETIME` para strings formatadas no padrão civil (DD/MM/AAAA) usando a função auxiliar `FileTimeToSystemTime`.
- **Thumbprint (Impressão Digital):** Calculado de forma unívoca a partir da extração da propriedade de hash criptográfico `CERT_SHA1_HASH_PROP_ID` via método `CertGetCertificateContextP`.
- **Has Private Key (Presença de Chave Privada):** Verificado logicamente ao tentar consultar o metadado de provimento de chave `CERT_KEY_PROV_INFO_PROP_ID`. Caso a função retorne sucesso, confirma-se que o par privado correspondente àquela chave pública está devidamente instalado e acessível no provedor criptográfico local da máquina.

5 Discussão sobre Segurança e Limitações

A arquitetura proposta foi desenhada sob a premissa de privilégio mínimo para sanar as restrições inerentes do modelo de segurança dos navegadores comerciais.

A principal barreira de segurança contornada reside no isolamento total imposto às páginas web. Se navegadores permitissem livre acesso do ecossistema JavaScript ao *Certificate Store* nativo, páginas maliciosas poderiam realizar varreduras em lote invisíveis, rastreando perfis digitais de usuários e coletando impressões digitais de identificação para fins de vigilância ou falsificação de sessão.

Para garantir conformidade com o Requisito Não Funcional de Segurança (RNF2), o componente nativo em C++ atua estritamente como um resolvidor de leitura de dados de nível público. O programa extrai apenas strings descritivas e hashes de verificação. As **chaves privadas associadas aos certificados jamais são lidas, exportadas ou trafegadas** pelos canais de transmissão do Native Messaging. Desse modo, mesmo na eventualidade hipotética de a extensão sofrer um ataque de injeção de script (XSS), nenhum vetor malicioso conseguirá comprometer a integridade das chaves privadas do usuário, as quais permanecem seladas pelo subsistema de proteção de credenciais do Windows.